

LAPORAN TUGAS: PENGEMBANGAN DAN ANALISIS SISTEM DISTRIBUSI PESAN & FILE BERBASIS HYBRID SOCKET

Nama: Sinta Siti Nuriah

Institusi: Universitas Gadjah Mada

Mata Kuliah: Jaringan Komputer Lanjut (S2 Ilmu Komputer)

Link GitHub: [https://github.com/Sintasitinuriah/Hybrid-Socket-System.git]

Bagian A: Implementasi Pemrograman Soket (Bobot 40%)

1. Layanan TCP Multi-Client (20%)

Potongan Kode:

```
1 serverSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
2 serverSocket.bind(('', serverPort))
3 serverSocket.listen(5)
4 while True:
5     connectionSocket, addr = serverSocket.accept()
6     client_thread = threading.Thread(target=handle_client, args=(
7         connectionSocket, addr))
8     client_thread.start()
```

Penjelasan: Sistem menangani beberapa klien secara konkuren dengan membuat *thread* baru melalui modul `threading` setiap kali `serverSocket.accept()` dipanggil.

Perbedaan Lifecycle: *Welcoming socket* (`serverSocket`) hidup terus-menerus selama server aktif untuk mendengarkan permintaan masuk di port 12000. Sedangkan *connection socket* (`connectionSocket`) dibuat khusus per sesi klien dan akan segera ditutup setelah klien terputus.

2. Layanan UDP Pinger (20%)

Potongan Kode:

```
1 clientSocket.settimeout(1.0)
2 for sequence_number in range(1, 11):
3     send_time = time.time()
4     try:
5         clientSocket.sendto(pesan.encode(), (serverName, serverPort))
6         modifiedMessage, serverAddress = clientSocket.recvfrom(1024)
7         rtt = (time.time() - send_time) * 1000
8     except socket.timeout:
9         print(f"Request {sequence_number} timed out")
```

Penjelasan: Fungsi `settimeout(1.0)` memaksa klien menunggu maksimal 1 detik. Jika paket di-drop (simulasi 40% loss di server), program akan memunculkan `socket.timeout`, mengabaikan iterasi tersebut, dan lanjut ke urutan *ping* berikutnya.

Bagian B: Analisis Mendalam & Arsitektur Jaringan (Bobot 60%)

1. Byte-Stream vs Message Boundary (15%)

- **Konsep:** TCP (SOCK_STREAM) adalah saluran *byte-stream* yang tidak mempedulikan batas pesan aplikasi, sedangkan UDP (SOCK_DGRAM) membungkus data dalam datagram yang menjaga batas pesan.
- **Dampak Teknis TCP:** Jika 3 string dikirim berturut-turut tanpa jeda, OS pada sisi penerima berpotensi menggabungkannya dalam satu *buffer* bacaan `recv()`.
- **Rancangan Protokol:** Untuk mengatasinya, protokol aplikasi yang saya buat menyisipkan karakter pemisah (*delimiter*) berupa *newline* (`\n`) pada setiap akhir pesan, sehingga server dapat memecah (*split*) aliran byte secara presisi.

2. Analisis Skalabilitas Socket (15%)

- **Total Soket TCP:** Untuk melayani N klien konkuren, server membutuhkan $N + 1$ soket (1 *welcoming socket* dan N *connection socket*).
- **Soket Tunggal UDP:** Server UDP hanya membutuhkan 1 soket karena bersifat *connectionless*. Server membaca datagram masuk lalu merespons menggunakan informasi *IP:Port* pengirim yang tersemat pada datagram.
- **Implikasi:** UDP jauh lebih hemat *file descriptor* OS dan memori dibandingkan TCP.

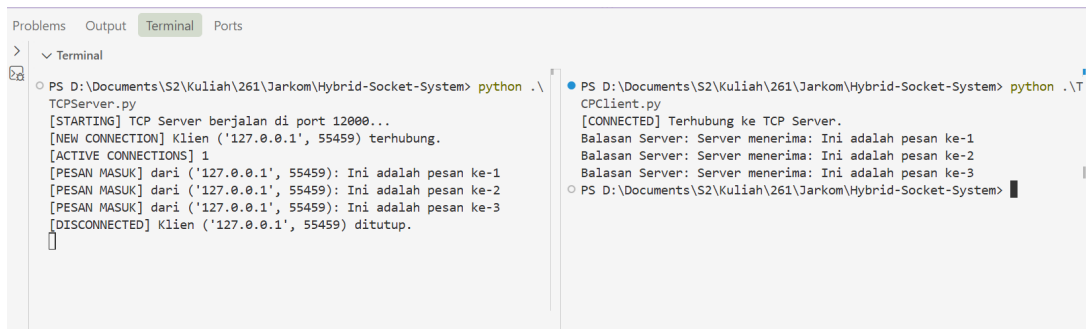
3. Evolusi Transport & QUIC (15%)

- **Alasan QUIC Memilih UDP:** Protokol baru di atas IP akan diblokir oleh berbagai *middlebox* (firewall/NAT) di internet. UDP bertindak sebagai landasan fleksibel agar logika QUIC dapat diimplementasikan cepat di *user-space*.
- **Mengatasi HOL Blocking:** QUIC memisahkan aliran data ke dalam *multiplexing stream* independen. Jika satu paket hilang, hanya *stream* yang terkait yang tertahan (bukan keseluruhan koneksi seperti pada TCP).

4. Skenario Bind Explicit pada Client (15%)

- **Kondisi `bind('', 5432)`:** Pesan UDP akan tetap terkirim, dan server dapat merespons kembali ke port 5432 tersebut dengan sukses.
- **Potensi Konflik:** Apabila terdapat 2 aplikasi klien yang dijalankan secara bersamaan pada komputer yang sama, instansi kedua akan mengalami *crash* (Address in Use) karena sistem operasi menolak pengikatan pada port lokal yang sama.

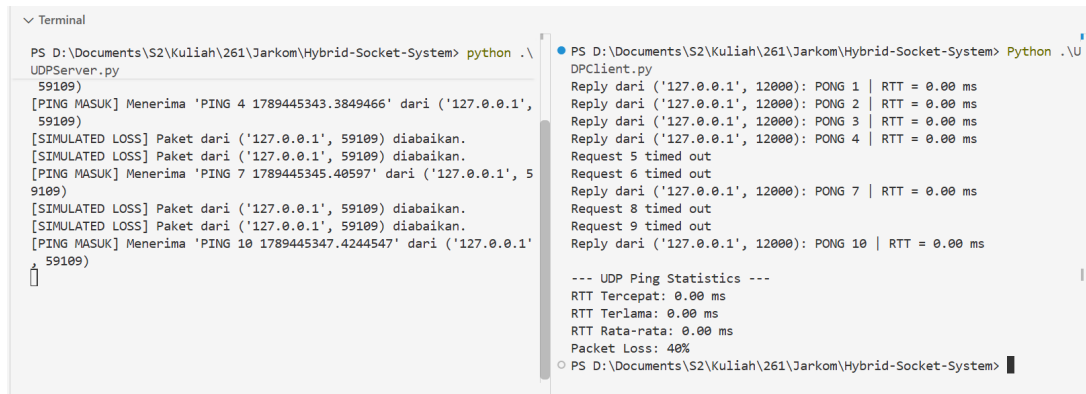
Lampiran: Tangkapan Layar Wireshark & Terminal



```
PS D:\Documents\S2\Kuliah\261\Jarkom\Hybrid-Socket-System> python .\TCPServer.py
[STARTING] TCP Server berjalan di port 12000...
[NEW CONNECTION] Klien ('127.0.0.1', 55459) terhubung.
[ACTIVE CONNECTIONS] 1
[PESAN MASUK] dari ('127.0.0.1', 55459): Ini adalah pesan ke-1
[PESAN MASUK] dari ('127.0.0.1', 55459): Ini adalah pesan ke-2
[PESAN MASUK] dari ('127.0.0.1', 55459): Ini adalah pesan ke-3
[DISCONNECTED] Klien ('127.0.0.1', 55459) ditutup.

PS D:\Documents\S2\Kuliah\261\Jarkom\Hybrid-Socket-System> python .\CPClient.py
[CONNECTED] Terhubung ke TCP Server.
Balasan Server: Server menerima: Ini adalah pesan ke-1
Balasan Server: Server menerima: Ini adalah pesan ke-2
Balasan Server: Server menerima: Ini adalah pesan ke-3
PS D:\Documents\S2\Kuliah\261\Jarkom\Hybrid-Socket-System>
```

Figure 1: Tangkapan Layar Terminal: Pengujian TCP Multi-Client

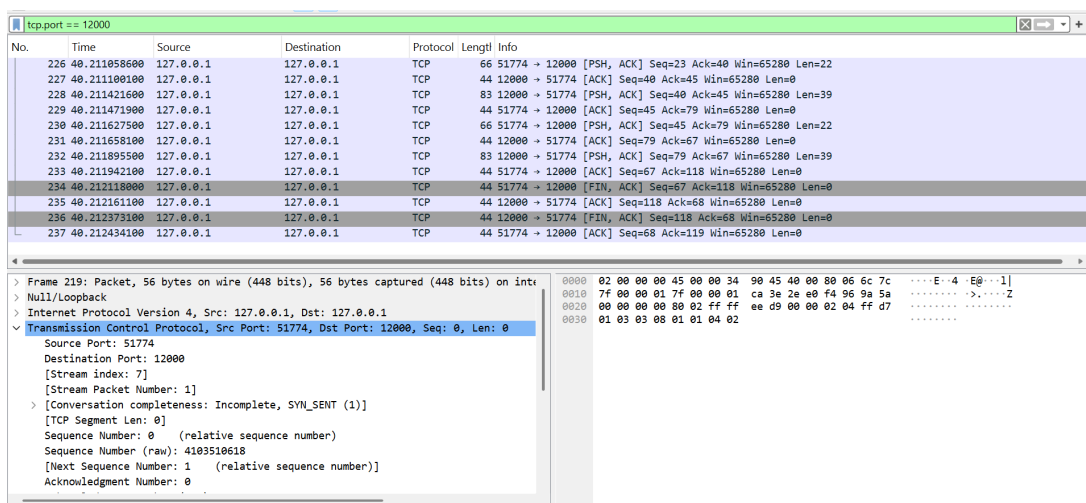


```
PS D:\Documents\S2\Kuliah\261\Jarkom\Hybrid-Socket-System> python .\UDPServer.py
59109
[PING MASUK] Menerima 'PING 4 1789445343.3849466' dari ('127.0.0.1', 59109)
[SIMULATED LOSS] Paket dari ('127.0.0.1', 59109) diabaikan.
[SIMULATED LOSS] Paket dari ('127.0.0.1', 59109) diabaikan.
[PING MASUK] Menerima 'PING 7 1789445345.40597' dari ('127.0.0.1', 59109)
[SIMULATED LOSS] Paket dari ('127.0.0.1', 59109) diabaikan.
[SIMULATED LOSS] Paket dari ('127.0.0.1', 59109) diabaikan.
[PING MASUK] Menerima 'PING 10 1789445347.4244547' dari ('127.0.0.1', 59109)

PS D:\Documents\S2\Kuliah\261\Jarkom\Hybrid-Socket-System> python .\DPCClient.py
Reply dari ('127.0.0.1', 12000): PONG 1 | RTT = 0.00 ms
Reply dari ('127.0.0.1', 12000): PONG 2 | RTT = 0.00 ms
Reply dari ('127.0.0.1', 12000): PONG 3 | RTT = 0.00 ms
Request 5 timed out
Request 6 timed out
Reply dari ('127.0.0.1', 12000): PONG 7 | RTT = 0.00 ms
Request 8 timed out
Request 9 timed out
Reply dari ('127.0.0.1', 12000): PONG 10 | RTT = 0.00 ms

--- UDP Ping Statistics ---
RTT Tercepat: 0.00 ms
RTT Terlama: 0.00 ms
RTT Rata-rata: 0.00 ms
Packet Loss: 40%
PS D:\Documents\S2\Kuliah\261\Jarkom\Hybrid-Socket-System>
```

Figure 2: Tangkapan Layar Terminal: Pengujian UDP Multi-Client



No.	Time	Source	Destination	Protocol	Length	Info
226	40.211058600	127.0.0.1	127.0.0.1	TCP	66	51774 → 12000 [PSH, ACK] Seq=23 Ack=40 Win=65280 Len=22
227	40.211180100	127.0.0.1	127.0.0.1	TCP	44	12000 → 51774 [ACK] Seq=40 Ack=45 Win=65280 Len=0
228	40.211421600	127.0.0.1	127.0.0.1	TCP	83	12000 → 51774 [PSH, ACK] Seq=40 Ack=45 Win=65280 Len=39
229	40.211471900	127.0.0.1	127.0.0.1	TCP	44	51774 → 12000 [ACK] Seq=45 Ack=79 Win=65280 Len=0
230	40.211627500	127.0.0.1	127.0.0.1	TCP	66	51774 → 12000 [PSH, ACK] Seq=45 Ack=79 Win=65280 Len=22
231	40.211658100	127.0.0.1	127.0.0.1	TCP	44	12000 → 51774 [ACK] Seq=79 Ack=67 Win=65280 Len=0
232	40.211895500	127.0.0.1	127.0.0.1	TCP	83	12000 → 51774 [PSH, ACK] Seq=79 Ack=67 Win=65280 Len=39
233	40.211942100	127.0.0.1	127.0.0.1	TCP	44	51774 → 12000 [ACK] Seq=67 Ack=118 Win=65280 Len=0
234	40.212118000	127.0.0.1	127.0.0.1	TCP	44	51774 → 12000 [FIN, ACK] Seq=67 Ack=118 Win=65280 Len=0
235	40.212161100	127.0.0.1	127.0.0.1	TCP	44	12000 → 51774 [ACK] Seq=118 Ack=68 Win=65280 Len=0
236	40.212373100	127.0.0.1	127.0.0.1	TCP	44	12000 → 51774 [FIN, ACK] Seq=118 Ack=68 Win=65280 Len=0
237	40.212434100	127.0.0.1	127.0.0.1	TCP	44	51774 → 12000 [ACK] Seq=68 Ack=119 Win=65280 Len=0

```
> Frame 219: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bits) on int
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 51774, Dst Port: 12000, Seq: 0, Len: 0
  Source Port: 51774
  Destination Port: 12000
  [Stream index: 7]
  [Stream Packet Number: 1]
  [Conversation completeness: Incomplete, SYN_SENT (1)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 4103510618
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
```

Figure 3: Tangkapan Layar Wireshark: TCP Three-Way Handshake

udp.port == 12000								
No.	Time	Source	Destination	Protocol	Length	Info		
1059	200.900318800	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP 0x48 Response		
1060	200.900791000	127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP EIA RS-511 Manufacturing Message Service Respon		
1061	200.901049100	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP 0x48 Response		
1062	200.901297800	127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP EIA RS-511 Manufacturing Message Service Respon		
1063	200.901560200	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP 0x48 Response		
1064	200.901849600	127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP EIA RS-511 Manufacturing Message Service Respon		
1065	200.902052900	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP 0x48 Response		
1066	200.902264100	127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP EIA RS-511 Manufacturing Message Service Respon		
1067	200.902421500	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP 0x48 Response		
1068	200.902646200	127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP EIA RS-511 Manufacturing Message Service Respon		
1069	200.902885000	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP 0x48 Response		
1070	200.903132900	127.0.0.1	127.0.0.1	LLC	38	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP EIA RS-511 Manufacturing Message Service Respon		
1071	200.903374700	127.0.0.1	127.0.0.1	LLC	57	I P, N(R)=35, N(S)=39; DSAP 0x50 Individual, SSAP 0x48 Response		

> Frame 1059: Packet, 57 bytes on wire (456 bits), 57 bytes captured (456 bits) on interface		0000	02 00 00 00 45 00 00 35 93 00 00 00 00 11 a9 b5	E..5
> Null/Loopback		0010	7f 00 00 01 7f 00 00 01 e7 51 2e e0 00 21 28 e2	Q..-l(
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1		0020	50 49 4e 47 20 31 20 31 37 38 39 34 34 35 39 38	PING 1.1 78944598
0100 = Version: 4		0030	37 2e 34 39 33 31 35 30 32	7.493150 2
.... 0101 = Header Length: 20 bytes (5)				
> Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)				
Total Length: 53				
Identification: 0x9300 (37632)				
> 0000 = Flags: 0x0				
...0 0000 0000 0000 = Fragment Offset: 0				
Time to Live: 128				
Protocol: UDP (17)				
Header Checksum: 0xa9b5 [validation disabled]				
[Header checksum status: Unverified]				

Figure 4: Tangkapan Layar Wireshark: Pertukaran Datagram UDP